Java lab2:字符串、数组和文件I/O

- 字符串
- 数组
- 文件I/O

字符串

创建字符串

声明和赋值

```
String str;
str = "hello";
```

或者，将声明与赋值在同一行代码完成

```
String str = "hello";
```

创建字符串的方式类似于基本数据类型，但注意，**String不是基础数据类型（primitive type）**

```
int a = 10;
double b = 3.14;
```

- 语法类似：都能直接用 = 快速赋值，例如 String str="hello" 看起来就像 int a=10 。
- 本质区别：String 属于引用数据类型（对象），不仅自带 length() 等专属方法，而且变量名中存放的是内存地址，并非单纯的数值。

字符串

字符串操作

1.length(): 获取字符串的长度

```
String str = "hello";
System.out.println(str.length()); //5
```

中文字符串同理

```
String str = "你好";
System.out.println(str.length()); //2
```

2.concat(): 拼接字符串

```
String str1 = "hello";
String str2 = ",world";
System.out.println(str1.concat(str2)); //helo,world
```

除了concat()之外，java还支持 `+`操作符拼接字符串

```
System.out.println(str1+str2);
```

字符串

字符串操作

`concat()`和`+`操作符的区别:

```
int a = 10;
System.out.println(str1.concat(a)); //编译错误
System.out.println(str1 + a); //hello10
```

`concat()`两侧都必须是String，不能是其他类型或者null。

对比项	concat()方法	+操作符
参数类型	只能接收String	可接收任意类型（自动转换）
空值处理	参数为null时报错	将null转为"null"字符串
性能	参数为空时返回原对象	每次连接都创建新对象
适用场景	确定参数为非空字符串	字符串与数字/布尔值混合连接

字符串

字符串操作

3.`equals()`：字符串的比较

```
String str1 = "hello";
String str2 = "hello";
String str3 = "world";
System.out.println(str1.equals(str2)); //true
System.out.println(str1.equals(str3)); //false
```

`equals()`用于比较字符串的内容（值比较），而不是引用(str1,str2是否指向同一个字符串)，可以使用`=`判断（引用比较）。

```
System.out.println(str1 = str2); //true
```

字符串

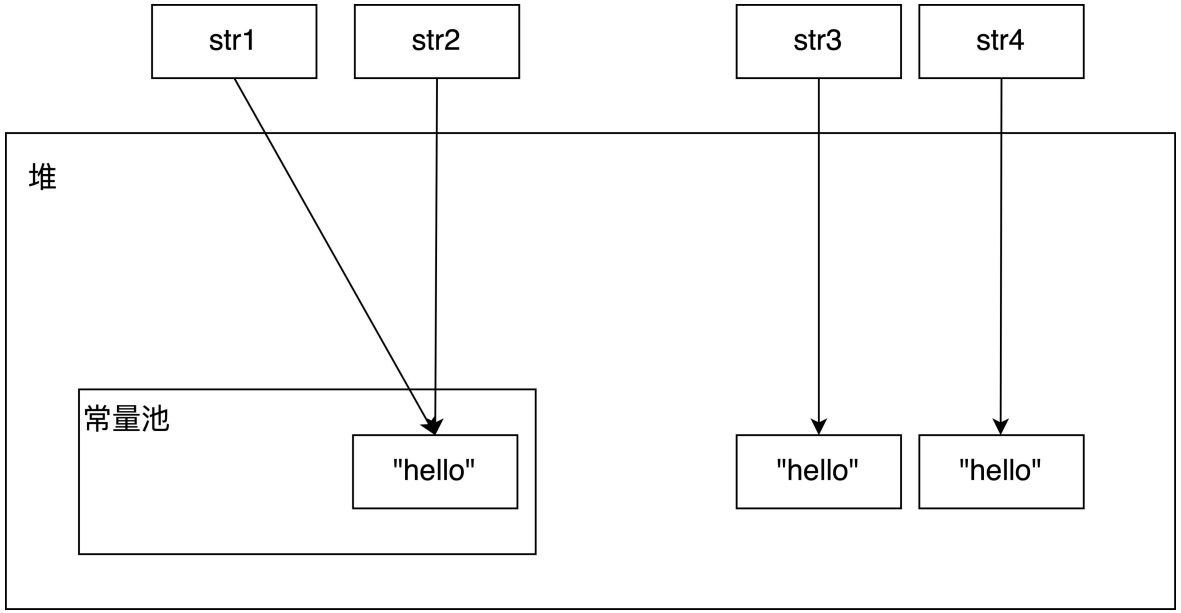
字符串操作

使用`new`关键字创建字符串

```
String str1 = "hello";
String str2 = "hello";
String str3 = new String("hello");
String str4 = new String("hello");
```

直接赋值创建的字符串存储在常量池中，而构造函数创建的字符串对象则存储在堆中。

对于内容相同的字符串，在常量池中永远只有一份，在堆中有多份。



StringBuilder

频繁拼接的代价

回顾我们刚刚学过的 concat() 和 + 拼接符 。

由于 String 的内容是不可变的，如果我们在一个执行 10000 次的 for 循环中使用 str = str + "a"，Java 会在内存（堆）中笨拙地创建和丢弃 10000 个临时字符串对象！

这将导致严重的性能下降和执行时间暴增。

StringBuilder是 Java 提供的一个“可变字符序列”。你可以把它看作一个可以随时扩容的内部缓冲区。

```
// ❌ 慢速且耗内存的做法（尽量避免在循环中使用）
String s = "";
for (int i = 0; i < 1000; i++) {
    s += "a";
}

// ✅ 现代 Java 推荐做法（极速）
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append("a"); // 全程只操作 sb 这一个对象
}
String result = sb.toString();
```

字符串

字符串操作

`equals()`与`=`操作符

比较方式	`equals()`方法	`==`操作符
比较内容	比较**字符串的实际内容**是否相同	比较**内存地址**（是否是同一个对象）
判断依据	"abc" 和 "abc" → true	同一个对象 → true；不同对象 → false

数组

一维数组

声明数组

```
int[] arr1; //推荐
int arr2[];
```

初始化数组

```
int[] arr1 = new int[10];
int[] arr2 = {1,2,3,4,5};
```

与C语言类似的是，数组被创建后，其长度不能被修改。

与C语言不同的是，数组初始化存在默认值（`arr1`数组的每个元素默认都是0）

数组

二维数组

声明数组

```
int[][] arr1; //推荐
int arr2[][];
```

初始化数组

```
// 方式1：矩形数组 - 固定行列
int[][] arr1 = new int[2][3];          // 2行3列，默认0
```

```
// 方式2：不规则数组 - 先定行数，后定每行列数
int[][] arr2 = new int[2][];           // 必须指定行数！
arr2[0] = new int[3];                  // 第一行3列
arr2[1] = new int[2];                  // 第二行2列
```

```
// 方式3：静态初始化（简化版）- 只能在声明时用
int[][] arr3 = {{1, 2, 3}, {4, 5}};    // 不规则数组
```

```
// 方式4：静态初始化（完整版）- 可重新赋值
int[][] arr4 = new int[][]{{1, 2}, {3, 4, 5}};
arr4 = new int[][]{{7, 8}, {9, 10}};   // ✅ 可重新赋值
// arr4 = {{11,12},{13,14}};           // ❌ 错误：{{}}只能在声明时用
```

```
// 方式5：逐行初始化 - 最灵活
int[][] arr5 = new int[3][];
arr5[0] = new int[]{1, 2, 3};          // 每行长度可不同
arr5[1] = new int[]{4, 5};
arr5[2] = new int[]{6};
```

数组

Java ArrayList

`ArrayList`是Java中的动态数组

```java
// 创建ArrayList集合，指定存储String类型
List<String> arr = new ArrayList<>();
arr.add("first");      // 添加元素"first"
arr.add("second");     // 添加元素"second"
arr.add("third");      // 添加元素"third"

// 方式1：普通for循环遍历
for (int i = 0; i < arr.size(); i++) {      // arr.size()获取集合长度
    System.out.println(arr.get(i));          // arr.get(i)获取第i个元素
}

// 方式2：增强for循环遍历（更简洁）
for (String str : arr) {                    // 遍历集合每个元素
    System.out.println(str);                 // 直接输出元素
}

arr.remove("second");     // 删除元素"second"
System.out.println(arr);  // 输出整个集合，格式：[first, third]
```

使用前需要`import java.util.List`和`import java.util.ArrayList`

数组

Java ArrayList和普通数组

对比项	普通数组	ArrayList
长度	固定，创建后不可变	动态，可自动扩容
类型	可存基本类型(int, char等)	只能存对象(需用Integer等包装类)
导入	不需要	需要 `import java.util.ArrayList;`
添加元素	`arr[0] = value;`	`list.add(value);`
访问元素	`arr[0]`	`list.get(0)`
删除元素	不支持	`list.remove(0);`
长度获取	`arr.length`	`list.size()`
性能	更快，内存更省	稍慢，有额外开销

文件I/O

流

- 流(Stream)：一组有顺序、有起点和终点的字节集合，是对数据传输的总称。也就是说，数据在两个对象之间的传输称为流。
- 按照流的传输方法，可以分为**输入流**和**输出流。**

程序->文件　输出

文件->程序　输入

- 流的基本操作有读操作和写操作，从流中取得数据的操作称为读操作，向流中添加数据的操作称为写操作。对输入流只能进行读操作，对输出流只能进行写操作。

文件I/O

FileIO

方法	说明
`void writeStringToFile(String str, String fileName)`	将一个字符串**追加**地写入指定文件（支持中文）
`char getCharFromFile(int pos, String fileName)`	读取指定文件的**第pos个字符**（支持中文）
`String getLineFromFile(int pos, String fileName)`	读取指定文件的**第pos行**的字符串（支持中文）
`String[] getAllLinesFromFile(String fileName)`	读取指定文件的**所有行**，按顺序存入String数组返回（支持中文）

文件I/O

FileIO

方法	说明
`void writeStringToFile(String str, String fileName)`	将一个字符串**追加**地写入指定文件（支持中文）
`char getCharFromFile(int pos, String fileName)`	读取指定文件的**第pos个字符**（支持中文）
`String getLineFromFile(int pos, String fileName)`	读取指定文件的**第pos行**的字符串（支持中文）
`String[] getAllLinesFromFile(String fileName)`	读取指定文件的**所有行**，按顺序存入String数组返回（支持中文）

```java
// 1. 写入文件（追加）
writeStringToFile("Hello World", "test.txt");
writeStringToFile("你好，世界", "test.txt");

// 2. 读取第6个字符
char ch = getCharFromFile(6, "test.txt");
System.out.println("第6个字符：" + ch);

// 3. 读取第2行
String line = getLineFromFile(2, "test.txt");
System.out.println("第2行：" + line);

// 4. 读取所有行
String[] lines = getAllLinesFromFile("test.txt");
for (int i = 0; i < lines.length; i++) {
    System.out.println("第" + (i+1) + "行：" + lines[i]);
}
```

输出内容：

```
第6个字符：W
第2行：你好，世界
第1行：Hello World
第2行：你好，世界
```

附录

字符串常用方法

方法	描述
`int length()`	返回字符串的长度
`String toUpperCase()`	将串中字符变成大写
`String toLowerCase()`	将串中字符变成小写
`char charAt(int index)`	返回位置index处的字符
`String substring(int s, int e)`	返回从位置s到e的字符子串[s,e)
`String substring(int s)`	返回从位置s到末尾的字符子串
`int indexOf(String s)`	返回首次出现字符串s的位置
`int indexOf(String s, int i)`	返回在位置i之后首次出现s的位置
`String trim()`	返回一个新串，去除前后空白字符